STUDENT COURSE MANAGEMENT SYSTEM

1. Introduction

The Student Course Management System is a database project developed using SQL to efficiently manage student records, course details, and enrollment information.

This project focuses on implementing core database operations such as data retrieval, filtering, updating, and analysis. It also demonstrates how multiple tables can be connected using relationships and how meaningful insights can be generated using SQL queries.

2. Objectives of the Project

- To design and implement a relational database system
- To perform data operations using SQL queries
- To understand relationships between multiple tables
- To apply filtering, sorting, and grouping techniques
- To analyze data using aggregate functions and subqueries

3. Database Design

The project is based on three main tables:

- **Students Table** → Stores student information
- **Courses Table** → Stores course details
- **Enrollments Table** → Maintains relationships between students and courses

4. Database Creation

Database creation is the process of initializing a structured environment to store and manage data efficiently. It provides a foundation for organizing tables, establishing relationships, and performing various SQL operations.

Query:

```
CREATE DATABASE Student_SPIC;
USE Student_SPIC;
```

Explanation:

This query creates a database named *Student_SPIC* and selects it for further use.

5. Table Structure

Table structure defines the organization of data into rows and columns, where each table represents a specific entity. It includes attributes, data types, and constraints such as primary keys and foreign keys to maintain data integrity and relationships.

5.1 Students Table

Query:

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    city VARCHAR(30),
    email VARCHAR(50) NOT NULL UNIQUE
);
```

Explanation:

Stores student personal and contact details.
Primary key ensures uniqueness, and email is constrained to be unique.

5.2 Courses Table

Query:

```
CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50),
    fees INT
);
```

Explanation:

Stores course-related information including course name and fee structure.

5.3 Enrollments Table

Query:

```
CREATE TABLE Enrollments (
    enroll_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    enroll_date DATE,
    FOREIGN KEY (student_id) REFERENCES
Students(student_id),
    FOREIGN KEY (course_id) REFERENCES
Courses(course_id)
);
```

Explanation:

Acts as a bridge table to maintain relationships between students and courses.

6. Data Insertion

Data insertion is the process of adding records to the database to simulate real-world scenarios, enabling effective querying, analysis, and relationship management.

Explanation:

Sample data was inserted into all tables using `INSERT INTO` statements. This helps simulate real-world data for executing queries and analysis.

6.1 Inserting Data into Students Table

(A) Question:

Insert multiple student records into the Students table including their personal and contact details to build a comprehensive dataset.

Query:

```
insert into Students(student_id,name,age,city,email)
    values
  (1,"Aman Rai",19,"Mumbai","AmanRai1@gmail.com"),
  (2,"Pawan Prajapati",20,"Delhi","PawanPrajapati2@gmail.com"),
  (3,"Navin Yadav",22,"Pune","NavinYadav3@gmail.com"),
  (4,"Ankur Rajput",21,"Lucknow","AnkurRajput4@gmail.com"),
  (5,"Ansh Yadav",25,"Kolkata","AnshYadav5@gmail.com"),
  (6,"Belal Hussain",27,"Nashik","BelalHussain6@gmail.com"),
  (7,"Ayush Rai",20,"Bengaluru","AyushRai7@gmail.com"),
  (8,"Sahil Prajapati",23,"Ahmedabad","SahilPrajapati8@gmail.com"),
  (9,"Imran Khan",22,"Nashik","ImranKhan9@gmail.com"),
  (10,"Ansh Rajput",24,"Varanasi","AnshRajput10@gmail.com"),
  (11,"Mohit Yadav",20,"Pune","MohitYadav11@gmail.com"),
  (12,"Jai Singh",26,"Mumbai","JaiSingh12@gmail.com"),
  (13,"Vipin Vishwakarma",25,"Mumbai","VipinVishwakarma13@gmail.com"),
  (14,"Varish Ansari",19,"Hyderabad","VarishAnsari14@gmail.com"),
  (15,"Shivam Sahu",17,"Delhi","ShivamSahu15@gmail.com"),
  (16,"Nikhil Rai",24,"Kanpur","NikhilRai16@gmail.com"),
  (17,"Nitish Panday",22,"Patna","NitishPanday17@gmail.com"),
  (18,"Vivek Maurya",26,"Chennai","VivekMaurya18@gmail.com"),
  (19,"Aditya Gupta",19,"Kanpur","AdityaGupta19@gmail.com"),
  (20,"Prashant Yadav",18,"Delhi","PrashantYadav20@gmail.com"),
  (21,"Adarsh Tiwari",22,"Mumbai","AdarshTiwari21@gmail.com"),
  (22,"Alok Prajapati",24,"Kolkata","AlokPrajapati22@gmail.com"),
  (23,"Abhay Gound",20,"Hyderabad","AbhayGound23@gmail.com"),
```

(24,"Sachin Kumar",25,"Pune","SachinKumar24@gmail.com"),
(25,"Shivam Sharma",21,"Pune","Shivamsharma25@gmail.com"),
(26,"Rishab Rajbhar",23,"Delhi","RishabRajbhar26@gmail.com"),
(27,"Rohan Kumar",20,"Surat","RohanKumar27@gmail.com"),
(28,"Krishna Rai",26,"Chennai","KrishnaRai28@gmail.com"),
(29,"Harsh Rai",21,"Lucknow","HarshRai29@gmail.com"),
(30,"Aryan Rai",24,"Kanpur","AryanRai30@gmail.com");

Explanation:
This query uses bulk insertion to add 30 student records in a single statement, which improves performance compared to inserting rows individually.

Each record contains:

- Unique student_id (Primary Key)
- Student name
- age for demographic analysis
- city for location-based insights
- email with UNIQUE constraint to avoid duplication

This dataset helps in performing operations like filtering, grouping, and sorting based on different attributes.

6.2 Inserting Data into Courses Table

(A) Question:
Insert course details including course names and fees into the Courses table to define available learning programs.

Query:

```
insert into Courses(course_id,course_name,fees)
    values
    (101,"SQL",4000),
    (102,"Python",6000 ),
    (103,"JavaScript ",3500),
    (104,"Advanced Excel",5000),
    (105,"Power BI",7000),
    (106,"Tableau",6000),
    (107,"Data Analysis",15000),
    (108,"Data Science",30000),
    (109,"HTML",4000),
    (110,"MySQL",4000),
    (111,"Machine Learning",25000),
    (112,"AWS",15000),
    (113,"MS PowerPoint",2000),
    (114,"MongoDB",6000),
```

```
    (115,"DevOps",18000),
    (116,"Django",10000),
    (117,"MS Word",2500),
    (118,"MongoDB",6000),
    (119,"Power BI",7000),
    (120,"SQL",4000),
    (121,"Python",6000),
    (122,"Machine Learning",25000),
    (123,"AWS",15000),
    (124,"Advanced Excel",5000),
    (125,"Django",10000),
    (126,"Power BI",7000),
    (127,"HTML",4000),
    (128,"MS PowerPoint",2000),
    (129,"JavaScript",3500),
    (130,"DevOps",18000);
```

Explanation:

This query inserts multiple course records covering different domains such as:

- Programming (Python, JavaScript)
- Data-related fields (Data Science, Machine Learning)
- Tools (Power BI, Tableau)

The fees column is especially important for:

- Performing aggregation (MAX, MIN, AVG)
- Identifying high-value courses
- Applying filters in subqueries

The dataset includes both low-cost and high-cost courses, making it suitable for advanced analysis.

6.3 Inserting Data into Enrollments Table

(A) Question:

Insert enrollment records to establish relationships between students and courses along with enrollment dates

Query:

```
insert into
Enrollments(enroll_id,student_id,course_id,enroll_date)
    values
    (1,1,101,"2025-06-10"),
    (2,2,102,"2025-07-12"),
    (3,3,103,"2025-09-15"),
    (4,4,104,"2025-04-18"),
    (5,5,105,"2025-05-20"),
```

```
(6,6,106,"2025-08-22"),
(7,7,107,"2025-02-25"),
(8,8,108,"2025-04-28"),
(9,9,109,"2025-09-30"),
(10,10,110,"2025-07-02"),
(11,11,111,"2025-03-05"),
(12,12,112,"2025-07-08"),
(13,13,113,"2025-06-23"),
(14,14,114,"2025-08-12"),
(15,15,115,"2025-04-15"),
(16,16,116,"2025-09-18"),
(17,17,117,"2025-07-20"),
(18,18,118,"2025-03-22"),
(19,19,119,"2025-06-25"),
(20,20,120,"2025-09-28"),
(21,21,121,"2025-04-30"),
(22,22,122,"2025-08-02"),
(23,23,123,"2025-03-01"),
(24,24,124,"2025-06-08"),
(25,25,125,"2025-08-10"),
(26,26,126,"2025-09-12"),
(27,27,127,"2025-04-15"),
(28,28,128,"2025-05-18"),
(29,29,129,"2025-08-20"),
(30,30,130,"2025-04-22");
```

Explanation:

This table plays a crucial role in the database as it connects Students and Courses using foreign keys.

Each record includes:

- enroll_id (Primary Key)
- student_id (Foreign Key)
- course_id (Foreign Key)
- enroll_date for time-based analysis

This structure enables:

- JOIN operations
- Tracking which student enrolled in which course
- Performing multi-table queries

6.4 Data Integrity and Relationships

Explanation:

The inserted data maintains referential integrity, meaning:

- Every student_id in Enrollments exists in Students table
- Every course_id in Enrollments exists in Courses table

This ensures:

- No invalid data
- Accurate JOIN results
- Reliable analysis

6.5 Data Verification

(A) Question:

Verify whether all records have been successfully inserted into the respective tables

Query:

```
SELECT * FROM Students;
```

student_id	name	age	city	email
1	Aman Rai	19	Mumbai	AmanRai1@gmail.com
2	Pawan Prajapati	20	Delhi	PawanPrajapati2@gmail.com
3	Navin Yadav	22	Pune	NavinYadav3@gmail.com
4	Ankur Rajput	21	Lucknow	AnkurRajput4@gmail.com
5	Ansh Yadav	25	Kolkata	AnshYadav5@gmail.com
6	Belal Hussain	27	Nashik	BelalHussain6@gmail.com
7	Ayush Rai	20	Bengaluru	AyushRai7@.com
8	Sahil Prajapati	23	Ahmedabad	SahilPrajapati8@gmail.com

```
SELECT * FROM Courses;
```

course_id	course_name	fees
101	SQL	4000
102	Python	6000
103	JavaScript	3500
104	Advanced Excel	5000
105	Power BI	7000
106	Tableau	6000
107	Data Analysis	15000
108	Data Science	30000

```
SELECT * FROM Enrollments;
```

enroll_id	student_id	course_id	enroll_date
1	1	101	2025-06-10
2	2	102	2025-07-12
3	3	103	2025-09-15
4	4	104	2025-04-18
5	5	105	2025-05-20
6	6	106	2025-08-22
7	7	107	2025-02-25
8	8	108	2025-04-28

Explanation:

These queries are used to:

- Check data correctness
- Ensure no missing or incorrect entries
- Confirm that the database is ready for further operations

7. SQL Queries and Analysis

SQL queries are used to retrieve, manipulate, and analyze data stored in the database. This section demonstrates the use of various SQL operations such as filtering, sorting, joins, aggregation, and subqueries to extract meaningful insights and understand relationships between tables

7.1 Basic Data Retrieval

(A) Question:

Retrieve the student IDs and names of all students stored in the database.

Query:

```
SELECT student_id, name FROM Students;
```

student_id	name
1	Aman Rai
2	Pawan Prajapati
3	Navin Yadav
4	Ankur Rajput
5	Ansh Yadav
6	Belal Hussain
7	Ayush Rai
8	Sahil Prajapati

Explanation:

Used to display basic student information for quick reference.

(B) Question:

Retrieve enrollment IDs and their respective enrollment dates.

Query:

```
SELECT enroll_id, enroll_date FROM Enrollments;
```

enroll_id	enroll_date
1	2025-06-10
2	2025-07-12
3	2025-09-15
4	2025-04-18
5	2025-05-20
6	2025-08-22
7	2025-02-25
8	2025-04-28

Explanation:

This query focuses only on specific columns, making it easier to analyze enrollment timelines without displaying unnecessary data.

(C) Question:

Fetch the names of all available courses along with their respective fees.

Query:

```
SELECT course_name, fees FROM Courses;
```

course_name	fees
SQL	4000
Python	6000
JavaScript	3500
Advanced Excel	5000
Power BI	7000
Tableau	6000
Data Analysis	15000
Data Science	30000

Explanation:

Helps in understanding course offerings and their pricing.

7.2 Data Filtering

(A) Question:

Retrieve all student records whose names start with the letter 'A' to analyze name-based patterns.

Query:

```
SELECT * FROM Students WHERE name LIKE "A%";
```

student_id	name	age	city	email
1	Aman Rai	19	Mumbai	AmanRai1@gmail.com
4	Ankur Rajput	21	Lucknow	AnkurRajput4@gmail.com
5	Ansh Yadav	25	Kolkata	AnshYadav5@gmail.com
7	Ayush Rai	20	Bengaluru	AyushRai7@.com
10	Ansh Rajput	24	Varanasi	AnshRajput10@gmail.com
19	Aditya Gupta	19	Kanpur	AdityaGupta19@gmail.com
21	Adarsh Tiwari	22	Mumbai	AdarshTiwari21@gmail.com
22	Alok Prajapati	24	Kolkata	AlokPrajapati22@gmail.com

Explanation:

The LIKE operator is used for pattern matching.

(B) Question:

Retrieve all courses whose names start with the letter 'S'.

Query:

```
SELECT * FROM Courses WHERE course_name LIKE "S%";
```

course_id	course_name	fees
101	SQL	4000
120	SQL	4000
NULL	NULL	NULL

Explanation:

The LIKE operator is used here to filter records based on a pattern. This is helpful when searching data with partial information.

(C) Question:

Find students whose age falls between 22 and 25 to study a specific age group.

Query:

```
SELECT student_id, name, age
FROM Students
WHERE age BETWEEN 22 AND 25;
```

student_id	name	age
3	Navin Yadav	22
5	Ansh Yadav	25
8	Sahil Prajapati	23
9	Imran Khan	22
10	Ansh Rajput	24
13	Vipin Vishwakarma	25
16	Nikhil Rai	24
17	Nitish Panday	22

Explanation:

BETWEEN is used to filter values within a range.

(D) Question:

Find students whose age is greater than 18 and less than 21.

Query:

```
SELECT student_id, name, age
FROM Students
WHERE age < 21 AND age > 18;
```

student_id	name	age
1	Aman Rai	19
2	Pawan Prajapati	20
7	Ayush Rai	20
11	Mohit Yadav	20
14	Varish Ansari	19
19	Aditya Gupta	19
23	Abhay Gound	20
27	Rohan Kumar	20

Explanation:

This query uses the AND operator to apply multiple conditions, ensuring that only records satisfying both conditions are returned.

(E) Question:

Retrieve enrollments that occurred between two specific dates.

Query:

```
SELECT enroll_id, enroll_date
FROM Enrollments
WHERE enroll_date BETWEEN "2025-04-12" AND "2025-06-22";
```

enroll_id	enroll_date
1	2025-06-10
4	2025-04-18
5	2025-05-20
8	2025-04-28
15	2025-04-15
21	2025-04-30
24	2025-06-08
27	2025-04-15

Explanation:

The BETWEEN operator is used to filter records within a range. It is particularly useful for time-based analysis.

7.3 Sorting and Limiting Data

(A) Question:

Retrieve the five youngest students by sorting the data in ascending order of age.

Query:

```
SELECT * FROM Students ORDER BY age ASC LIMIT 5;
```

student_id	name	age	city	email
15	Shivam Sahu	17	Delhi	ShivamSahu15@gmail.com
20	Prashant Yadav	18	Delhi	PrashantYadav20@gmail.com
1	Aman Rai	19	Mumbai	AmanRai1@gmail.com
14	Varish Ansari	19	Hyderabad	VarishAnsari14@gmail.com
19	Aditya Gupta	19	Kanpur	AdityaGupta19@gmail.com
NULL	NULL	NULL	NULL	NULL

Explanation:

ORDER BY organizes data, and LIMIT restricts the number of records.

(B) Question:

Retrieve the top 10 courses with the highest fees.

Query:

```
SELECT * FROM Courses ORDER BY fees DESC LIMIT 10;
```

course_id	course_name	fees
108	Data Science	30000
111	Machine Learning	25000
122	Machine Learning	25000
115	DevOps	18000
130	DevOps	18000
107	Data Analysis	15000
112	AWS	15000
123	AWS	15000

Explanation:

This query sorts the courses in descending order based on fees and limits the output to the top 10 records.

(C) Question:

Retrieve the most recent 15 enrollment records.

Query:

```
SELECT * FROM Enrollments ORDER BY enroll_date DESC
LIMIT 15;
```

enroll_id	student_id	course_id	enroll_date
9	9	109	2025-09-30
20	20	120	2025-09-28
16	16	116	2025-09-18
3	3	103	2025-09-15
26	26	126	2025-09-12
6	6	106	2025-08-22
29	29	129	2025-08-20
14	14	114	2025-08-12

Explanation:

This helps in analyzing recent enrollment trends by showing the latest entries first.

(D) Question:

Display all students sorted alphabetically by their names.

Query:

```
SELECT * FROM Students ORDER BY name;
```

student_id	name	age	city	email
23	Abhay Gound	20	Hyderabad	AbhayGound23@gmail.com
21	Adarsh Tiwari	22	Mumbai	AdarshTiwari21@gmail.com
19	Aditya Gupta	19	Kanpur	AdityaGupta19@gmail.com
22	Alok Prajapati	24	Kolkata	AlokPrajapati22@gmail.com
1	Aman Rai	19	Mumbai	AmanRai1@gmail.com
4	Ankur Rajput	21	Lucknow	AnkurRajput4@gmail.com
10	Ansh Rajput	24	Varanasi	AnshRajput10@gmail.com
5	Ansh Yadav	25	Kolkata	AnshYadav5@gmail.com

Explanation:

Sorting data alphabetically improves readability and helps in quickly locating specific records.

7.4 Data Modification

(A) Question:

Update the name of a student from "Aman Rai" to "Amit Yadav" to maintain accurate records.

Query:

```
UPDATE Students
SET name = "Amit Yadav"
WHERE name = "Aman Rai";
select * from students
```

name	course_name
Amit Yadav	SQL
Pawan Prajapati	Python
Navin Yadav	JavaScript
Ankur Rajput	Advanced Excel
Ansh Yadav	Power BI
Belal Hussain	Tableau
Ayush Rai	Data Analysis
Sahil Prajapati	Data Science

Explanation:

The UPDATE statement is used to modify existing data.

(B) Question:

Update the course name for a specific course ID.

Query:

```
UPDATE Courses
SET course_name ="C++"
WHERE course_id=112;
select * from students
```

course_id	course_name	fees	fees_status
109	HTML	4000	NULL
110	MySQL	4000	NULL
111	Machine Learning	25000	NULL
112	C++	15000	NULL
113	MS PowerPoint	2000	NULL
114	MongoDB	6000	NULL
115	DevOps	18000	NULL
116	Django	10000	NULL

Explanation:

This query modifies existing course information, ensuring the data remains accurate and up-to-date.

(C) Question:

Update the enrollment date for a specific enrollment record.

Query:

```
UPDATE Enrollments
SET enroll_date ="2025-12-29"
WHERE enroll_id=5;
select * from Enrollments;
```

enroll_id	student_id	course_id	enroll_date
1	1	101	2025-06-10
2	2	102	2025-07-12
3	3	103	2025-09-15
4	4	104	2025-04-18
5	5	105	2025-12-29
6	6	106	2025-08-22
7	7	107	2025-02-25
8	8	108	2025-04-28

Explanation:

Used to correct or update incorrect date entries in the database.

7.5 Table Alter Operations

(A) Question:

Add a new column named fees_status to the Courses table.

Query:

```
ALTER TABLE Courses
ADD fees_status VARCHAR(20);
DESC Courses;
```

	Field	Type	Null	Key	Default	Extra
▶	course_id	int	NO	PRI	NULL	
	course_name	varchar(50)	YES		NULL	
	fees	int	YES		NULL	
	fees_status	varchar(20)	YES		NULL	

Explanation:

This query modifies the table structure by adding a new column, which can be used for additional data classification

(B) Question:

Display the structure of the Courses table.

Query:

```
DESC Courses;
```

	Field	Type	Null	Key	Default	Extra
▶	course_id	int	NO	PRI	NULL	
	course_name	varchar(50)	YES		NULL	
	fees	int	YES		NULL	
	fees_status	varchar(20)	YES		NULL	

Explanation:

This command shows column names, data types, and constraints.

(C) Question:

Rename the column email to email_id in the Students table.

Query:

```
ALTER TABLE Students RENAME COLUMN email TO email_id;
```

	Field	Type	Null	Key	Default	Extra
▶	student_id	int	NO	PRI	NULL	
	name	varchar(50)	YES		NULL	
	age	int	YES		NULL	
	city	varchar(30)	YES		NULL	
	email_id	varchar(50)	NO	UNI	NULL	

Explanation:

This improves naming consistency and clarity.

7.6 Aggregate Functions

(A) Question:

Determine the maximum course fee available in the database.

Query:

```
SELECT MAX(fees) FROM Courses;
```

	MAX(fees)
▶	30000

Explanation:

Aggregate functions perform calculations on data.

(B) Question:

Determine the minimum course fee available.

Query:

```
SELECT MIN(fees) FROM Courses;
```

	Min(fees)
▶	2000

Explanation:

The MIN() function returns the lowest value in the column, helping identify the least expensive course.

7.7 IN Operator

(A) Question:

Retrieve students who belong to Mumbai, Delhi, or Pune.

Query:

```
SELECT * FROM Students
WHERE city IN ("Mumbai","Delhi","Pune");
```

	student_id	name	age	city	email_id
▶	1	Amit Yadav	19	Mumbai	AmanRai1@gmail.com
	2	Pawan Prajapati	20	Delhi	PawanPrajapati2@gmail.com
	3	Navin Yadav	22	Pune	NavinYadav3@gmail.com
	11	Mohit Yadav	20	Pune	MohitYadav11@gmail.com
	12	Jai Singh	26	Mumbai	JaiSingh12@gmail.com
	13	Vipin Vishwakarma	25	Mumbai	VipinVishwakarma13@gmail.com
	15	Shivam Sahu	17	Delhi	ShivamSahu15@gmail.com
	20	Prashant Yadav	18	Delhi	PrashantYadav20@gmail.com

Explanation:

The IN operator allows checking multiple values in a single condition, making the query concise and efficient.

(B) Question:

Retrieve details of specific students based on their names.

Query:

```
SELECT * FROM Students
WHERE name IN ("Vipin Vishwakarma","Ankur
Rajput","Krishna Rai");
```

student_id	name	age	city	email_id
4	Ankur Rajput	21	Lucknow	AnkurRajput4@gmail.com
13	Vipin Vishwakarma	25	Mumbai	VipinVishwakarma13@gmail.com
28	Krishna Rai	26	Chennai	KrishnaRai28@gmail.com
NULL	NULL	NULL	NULL	NULL

Explanation:

This query is useful when searching for multiple specific records at once.

7.8 JOIN Operations

(A) Question:

Retrieve student names, course names, and enrollment dates where valid relationships exist across all tables.

INNER JOIN

Query:

```
SELECT s.name, c.course_name, e.enroll_date
FROM Students s
INNER JOIN Enrollments e ON s.student_id =
e.student_id
INNER JOIN Courses c ON e.course_id = c.course_id;
```

name	course_name	enroll_date
Amit Yadav	SQL	2025-06-10
Pawan Prajapati	Python	2025-07-12
Navin Yadav	JavaScript	2025-09-15
Ankur Rajput	Advanced Excel	2025-04-18
Ansh Yadav	Power BI	2025-12-29
Belal Hussain	Tableau	2025-08-22
Ayush Rai	Data Analysis	2025-02-25
Sahil Prajapati	Data Science	2025-04-28

Explanation:

Returns only matching records from all tables.

(B) Question:

Display all students along with their enrolled courses, including those students who have not enrolled in any course.

LEFT JOIN

Query:

```
SELECT s.name, c.course_name
FROM Students s
LEFT JOIN Enrollments e ON s.student_id =
e.student_id
LEFT JOIN Courses c ON e.course_id = c.course_id;
```

name	course_name
Amit Yadav	SQL
Pawan Prajapati	Python
Navin Yadav	JavaScript
Ankur Rajput	Advanced Excel
Ansh Yadav	Power BI
Belal Hussain	Tableau
Ayush Rai	Data Analysis
Sahil Prajapati	Data Science

Explanation:

Ensures all students are included.

(C) Question:

Retrieve all courses along with the students enrolled in them, including courses that have no enrollments.

RIGHT JOIN

Query:

```
SELECT s.name, c.course_name
FROM Students s
RIGHT JOIN Enrollments e ON s.student_id =
e.student_id
RIGHT JOIN Courses c ON e.course_id = c.course_id;
```

name	course_name
Amit Yadav	SQL
Pawan Prajapati	Python
Navin Yadav	JavaScript
Ankur Rajput	Advanced Excel
Ansh Yadav	Power BI
Belal Hussain	Tableau
Ayush Rai	Data Analysis
Sahil Prajapati	Data Science

Explanation:

Ensures all courses are included.

(D) Question:

Retrieve all records from both Students and Courses tables, including unmatched records.

FULL OUTER JOIN

Query:

```
SELECT s.name, c.course_name
FROM Students s
LEFT JOIN Enrollments e ON s.student_id =
e.student_id
LEFT JOIN Courses c ON e.course_id = c.course_id
```

```
UNION
SELECT s.name, c.course_name
FROM Students s
RIGHT JOIN Enrollments e ON s.student_id =
e.student_id
RIGHT JOIN Courses c ON e.course_id = c.course_id;
```

name	course_name
Amit Yadav	SQL
Pawan Prajapati	Python
Navin Yadav	JavaScript
Ankur Rajput	Advanced Excel
Ansh Yadav	Power BI
Belal Hussain	Tableau
Ayush Rai	Data Analysis
Sahil Prajapati	Data Science

Explanation:

Since MySQL does not support FULL OUTER JOIN directly, it is simulated using a combination of LEFT JOIN and RIGHT JOIN with UNION.

7.9 GROUP BY and HAVING

(A) Question:

Group students based on their city and calculate the total number of students in each city.

Query:

```
SELECT city, COUNT(*) AS total_students
FROM Students
GROUP BY city;
```

city	total_students
Mumbai	4
Delhi	4
Pune	4
Lucknow	2
Kolkata	2
Nashik	2
Bengaluru	1
Ahmedabad	1

Explanation:

Used to summarize data.

(B) Question:

Identify cities where the number of students exceeds three, using grouping and filtering.

Query:

```
SELECT city, COUNT(*) AS total_students
FROM Students
GROUP BY city
HAVING COUNT(*) > 3;
```

city	total_students
Mumbai	4
Delhi	4
Pune	4

Explanation:

HAVING filters grouped results.

7.10 JOIN with Condition (Counting)

(A) Question:

Count the number of students enrolled in the Power BI course.

Query:

```
SELECT COUNT(*) AS total_PowerBi_students
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id
WHERE c.course_name = "Power BI";
```

total_sql_students
3

Explanation:

This query uses JOIN to combine data from multiple tables and COUNT to calculate the total number of students meeting the condition.

7.11 JOIN with Date Condition

(A) Question:

Retrieve student enrollments that occurred after a specific date.

Query:

```
SELECT s.name, c.course_name, e.enroll_date
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id
WHERE e.enroll_date > "2025-01-01";
```

name	course_name	enroll_date
Amit Yadav	SQL	2025-06-10
Pawan Prajapati	Python	2025-07-12
Navin Yadav	JavaScript	2025-09-15
Ankur Rajput	Advanced Excel	2025-04-18
Ansh Yadav	Power BI	2025-12-29
Belal Hussain	Tableau	2025-08-22
Ayush Rai	Data Analysis	2025-02-25
Sahil Prajapati	Data Science	2025-04-28

Explanation:

This query combines multiple tables and filters results based on a date condition.

7.12 Subquery

(A) Question:

Retrieve the names of students who are enrolled in high-fee courses (greater than 10,000) using nested subqueries

Query:

```
SELECT name FROM Students
WHERE student_id IN (
    SELECT student_id FROM Enrollments
    WHERE course_id IN (
        SELECT course_id FROM Courses WHERE fees >
10000));
```

name
Ayush Rai
Sahil Prajapati
Mohit Yadav
Jai Singh
Shivam Sahu
Alok Prajapati
Abhay Gound
Aryan Rai

Explanation:

Subqueries allow complex filtering using multiple levels of conditions.

8. Conclusion

This project provided practical exposure to database management using SQL. It helped in understanding how structured data can be stored, retrieved, and analyzed efficiently.

The use of joins, grouping, and subqueries enhanced the ability to work with complex relational data, making this project valuable for real-world database applications.

9. Tools & Technologies Used

- Database: MySQL
- Language: SQL
- Concepts: Joins, Subqueries, Aggregation, Filtering, Sorting